

UGV-UAV

Jnanesh Sathisha Karmar, Vivek K Kumar,
Dr. G.V.V. Sharma
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

Abstract—This work aims to integrate UAV architecture into a UGV platform to develop a unified control system for both domains. This is efficient, and also enables testing of UAVs through UGVs, as effectively UAVs only have an extra dimension.

I. INTRODUCTION

Our goal is to take a drone with a standard multicopter architecture, and try to integrate each component, part by part into a ground vehicle. drone architecture and integrate its core components into a ground-based vehicle. Traditional UAVs rely on high-RPM brushless motors, Electronic Speed Controllers (ESCs), and specialized flight controllers. While components like the power delivery systems and radio transmitters are universally applicable, others—such as the motor control mechanisms—require adaptation for ground use

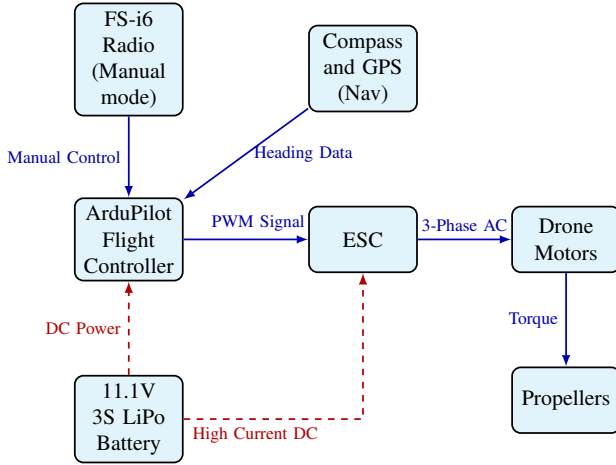


Fig. 1. Drone architecture

The FS-i6 radio system can be reliably used in both the drone framework and the ground vehicle framework. The range of this system is around 500m-1000m in clear air (without obstacles in between).

The same DC power source can be used for both frameworks. The 3S LiPo batteries used in the system are highly versatile.

Due to low torque and high RPM, we concluded that the drone motors cannot directly be used in a ground vehicle

framework. Though it might work for certain surfaces, we did not test the effectiveness of this approach. We instead use geared DC Motors (via a motor driver). Keeping this in mind, we integrate each of these components into the ground vehicle framework.

II. BACKGROUND & SYSTEM COMPONENTS

A. ArduPilot & Mission Planner

ArduPilot [?] is a versatile autopilot system for multicopters, rovers, boats, and planes. It reads sensors (gyro, GPS) to controls motors. One of its key advantages is it's ability to handle both high-speed flight stabilization and precise ground steering. This enables the same hardware to be used both as a flight controller and a ground vehicle controller.

Mission Planner [?] is a free, open-source Ground Control Station (GCS) software used to configure, monitor, and control autonomous vehicles such as drones, planes, rovers, and boats that run on ArduPilot firmware. It provides a graphical interface to communicate with the flight controller through USB or telemetry radios. Mission Planner allows users to upload flight missions, set waypoints, tune parameters, calibrate sensors, and monitor real-time flight data such as altitude, speed, GPS position, and battery status.



Fig. 2. Mission Planner interface

B. Flysky Radio System

The Flysky FS-i6 [?] is a 2.4 GHz wireless radio transmitter used for remote control of electronic systems such as drones, robots, RC cars, airplanes, and embedded systems. It is part of the Flysky radio control ecosystem and works together with compatible receivers (such as FS-IA6 or FS-IA6B) to transmit

user commands wirelessly to a microcontroller or motor control system. The FS-i6 converts physical inputs from joysticks, switches, and knobs into digital signals, which are transmitted using the AFHDS 2A (Automatic Frequency Hopping Digital System) protocol. The receiver connected to the robot or vehicle interprets these signals and generate corresponding control signals to motors, servos, or microcontrollers.



Fig. 3. flysky remote

III. UAV TO UGV INTEGRATION

A. Integration with L298N Motor Drivers

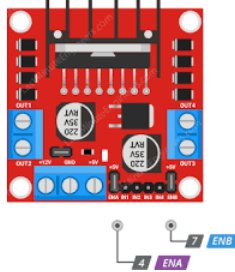


Fig. 4. L298N

The system architecture follows a sequential control flow from the APM 2.8 flight controller, through an ESP32 microcontroller, to an L298N motor driver [?], which ultimately powers the DC motors.

1) **The Signal Translation Challenge:** A direct connection between the APM and the motor driver is not feasible due to a fundamental signal mismatch. Specifically, the APM 2.8 outputs standard RC servo PWM pulses ranging from 1000 μ s to 2000 μ s, whereas the L298N driver requires logic-level directional inputs (HIGH/LOW) and standard duty-cycle PWM (0–100%) to regulate speed.

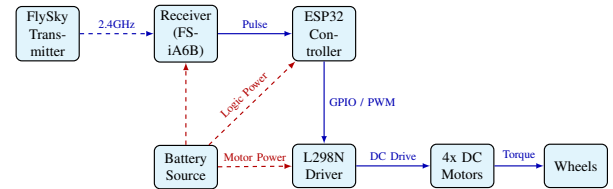
2) **ESP32 as a Middleware Controller:** To bridge this gap, the ESP32 functions as a dedicated signal translator. It continuously reads the incoming pulse widths from the APM's throttle and steering outputs and mathematically maps these values to the required motor driver formats. For example, the ESP32 is programmed to interpret a signal of 1000–1450 μ s as a reverse command, 1450–1550 μ s as a neutral deadband for stopping or braking, and 1550–2000 μ s as forward motion. Based on this real-time mapping, the ESP32 drives the appropriate pins on the L298N to achieve precise locomotion.

B. Setup

- 1) Open Mission Planner and flash the ArduRover v2.5.1 firmware onto the APM 2.8 flight controller. Custom C++ code is not required for the APM, as the ArduPilot ecosystem provides precompiled stacks in the ArduPilot website.
- 2) Calibrate radio, compass, GPS, accelerometer and servo output in Mission Planner
- 3) Wire APM 2.8 as follows:
 - Inputs: Connect radio receiver and GPS/Compass module. Also power it up through the 5V L298N motor driver line.
 - Outputs: Connect signal lines of 1 and 3 to ESP32.
- 4) Upload the code into the ESP32 from the following repository

<https://github.com/Vivek-Kumar-1907/blob/main/codes/main.ino>

C. Summary Architecture (FlySky)



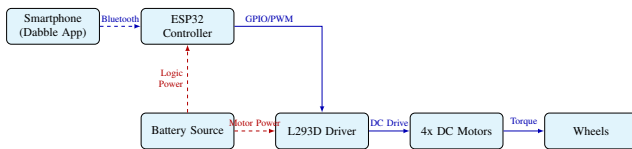
D. Result

We have hence successfully translated the entire UAV architecture into an equivalent UGV architecture.

APPENDIX

Dabble [?] is a mobile application developed by STEMpedia that enables smartphones to function as wireless controllers for microcontroller-based systems such as Arduino, ESP32, and other embedded platforms. It uses Bluetooth communication to connect the smartphone with the hardware, allowing users to control, monitor, and interact with electronic projects in real time. Dabble provides a set of pre-built modules that simplify interfacing between hardware and smartphone sensors, eliminating the need to design complex mobile applications from scratch. Through its graphical interface, users can access smartphone features such as buttons, sensors, camera input, and motion detection, and transmit that data directly to the microcontroller.

A. Component Architecture with Dabble



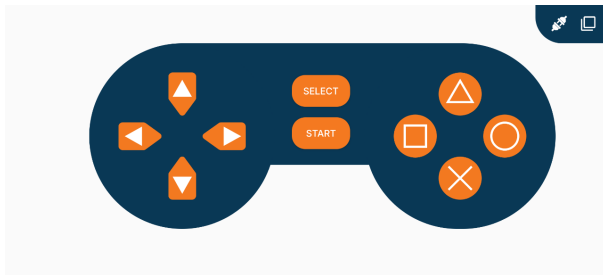
B. Setup and Operation

Setup & Connection:

- 1) Install **Dabble** app (Android).
- 2) Enable Bluetooth on your phone.
- 3) Open the app and select the **Gamepad** module.
- 4) Tap the **Plug Icon** (Connect) at the top right.
- 5) Select your ESP32 (e.g., "JNANU").

Controls:

- Use the **Digital Mode** (Arrow keys).
- **Up/Down**: Move Forward/Backward.
- **Left/Right**: Turn (Differential Drive).



Dabble Gamepad Interface